

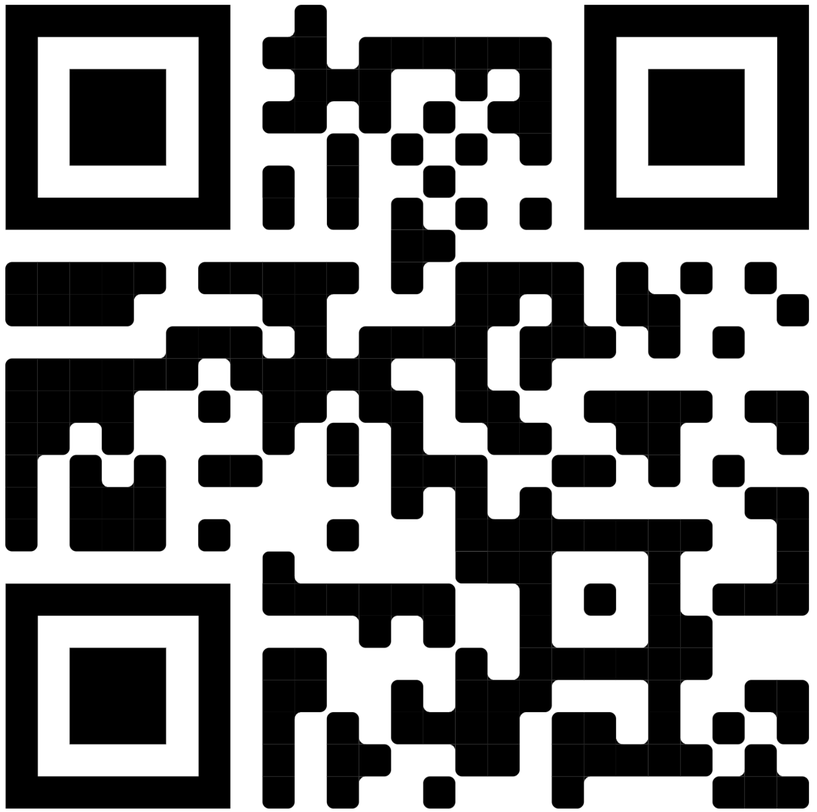


Distributed systems

Lectures: doc. dr. sc. Nikola Tanković

Introduction





http://bit.ly/rs-tu-sam

Course Goals

- Introduction to theoretical concepts of designing distributed systems
- Master developing simple distributed systems through several small projects

Practicum topics

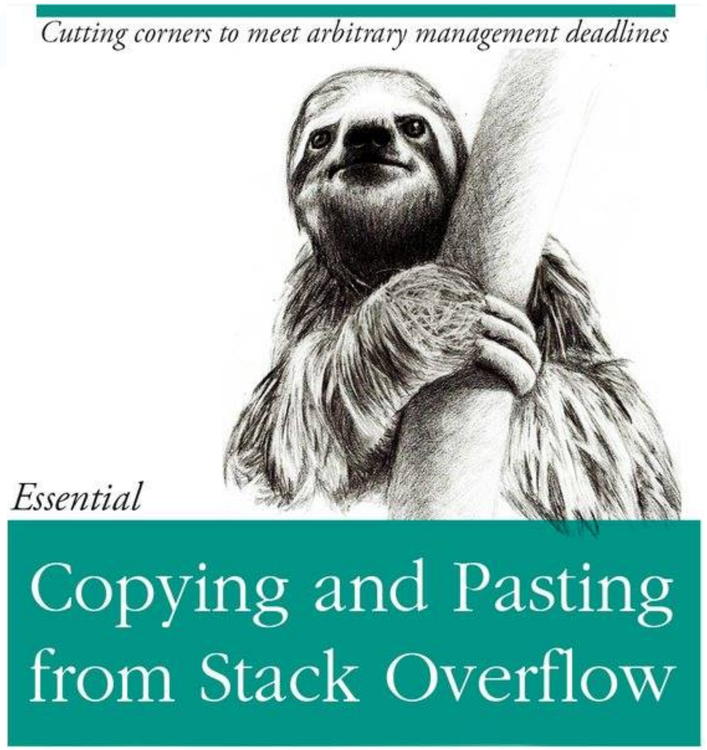
- **Microservice architecture** - implementing microservices in sync and async communication patterns
- **Big-data analysis** - Implementing parallel and distributed programs for data analysis

Students obligations

- Attending classes (lectures & practicum)
  - **not mandatory!**
  - but if so, please be active! 👍🏼
- Create **projects individually**
  - progress will be monitored on GitHub
- Defend the **projects** and pass the oral exam

Seeking help

1. Googling **Stackoverflow**
2. **Official** documentation
3. **Slack** group
   - Answers, directions...
     → **#trebam-pomoc**
4. Colleagues
5. Class Assistant
6. Consultations
   (Slack, mail, live)



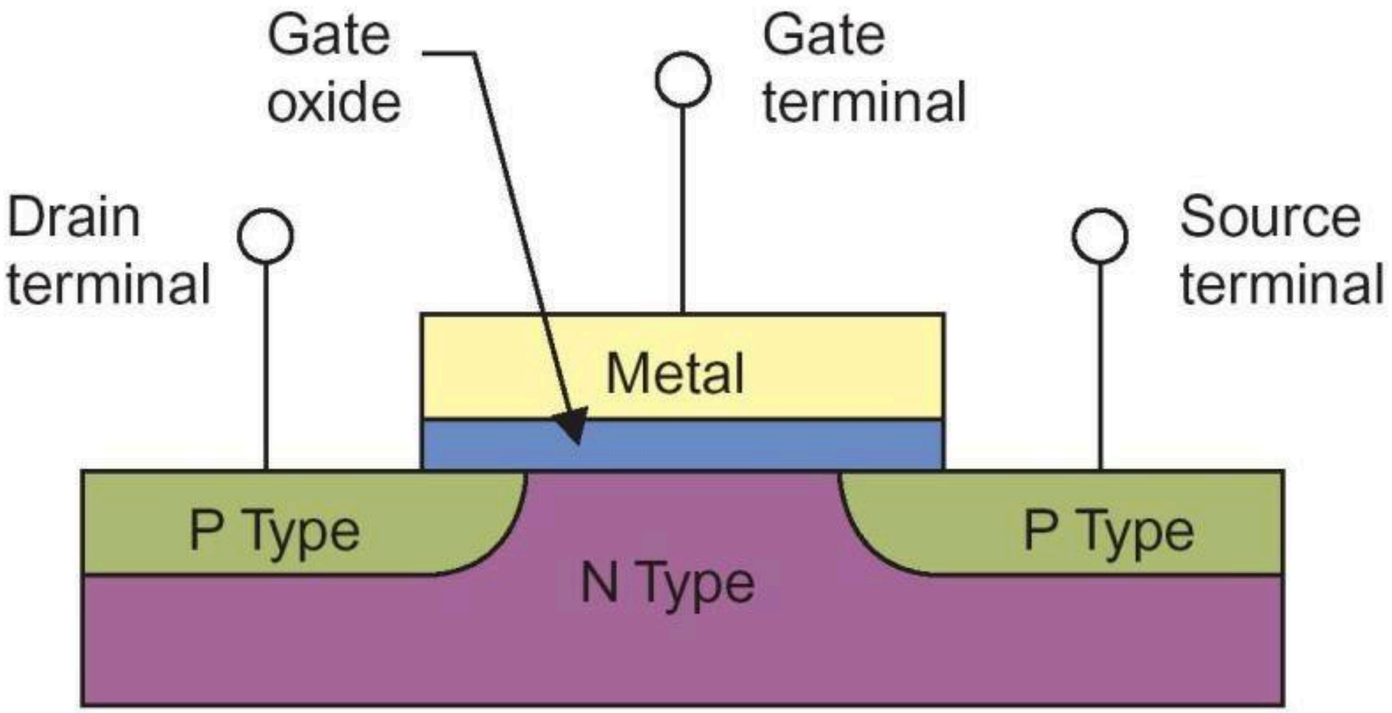




Distributed systems

Introduction and key definitions

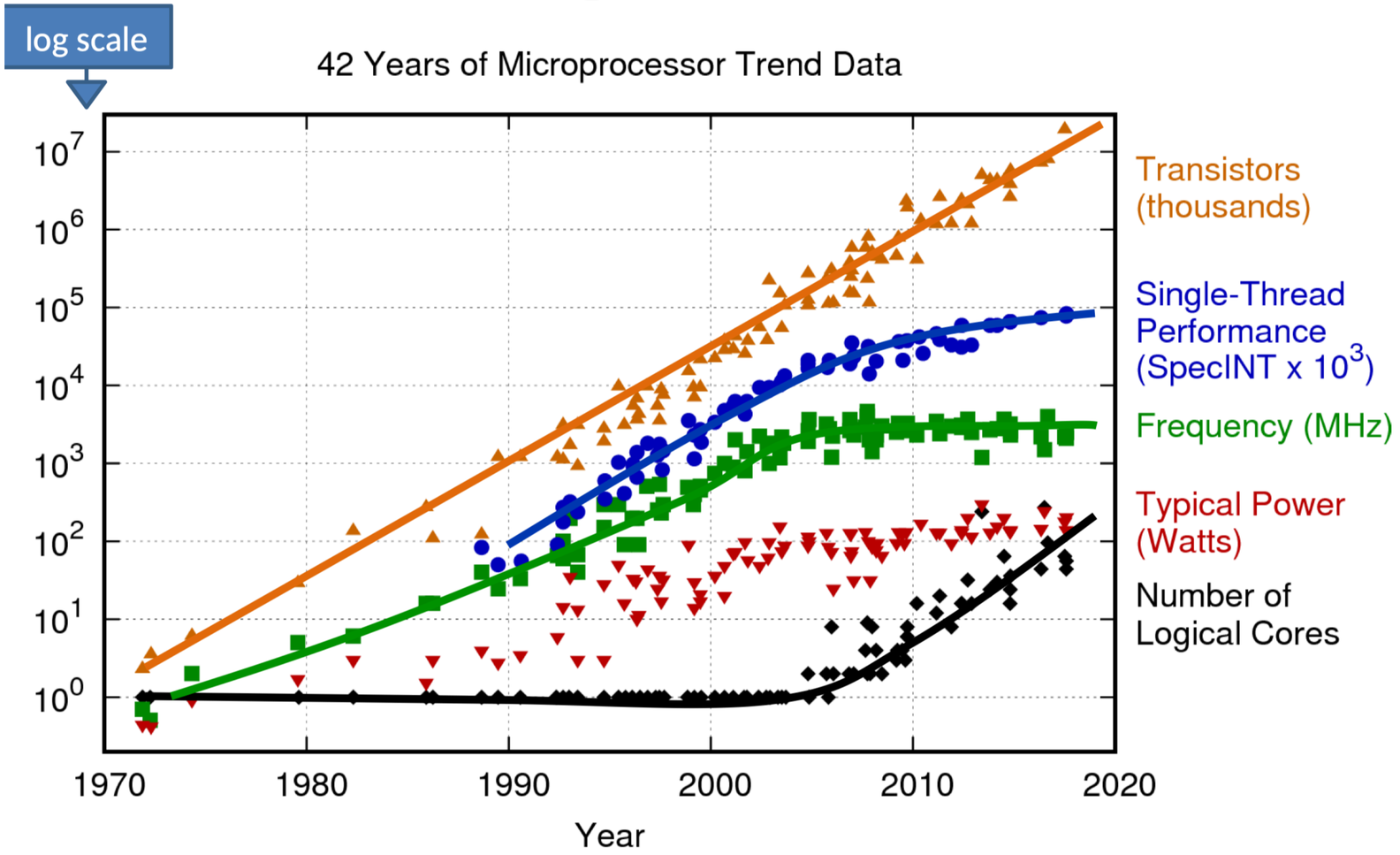
Is smaller possible?



Transistor limits

- 14 nm transistor production
  - just 70 silicon atoms wide
  - 14 times wider than DNA molecule
- Thermal limits
  - turning transistors on/off thousands of times per second
  - heat is deadly to silicon
- Impedance rises on wires connecting transistors

Key trends



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/

DEF

Distributed system

- **A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system.**

Key features:

– **Collection** of elements - hardware device or software process
  - Referred to as a ***node,*** sometimes also as an ***agent***

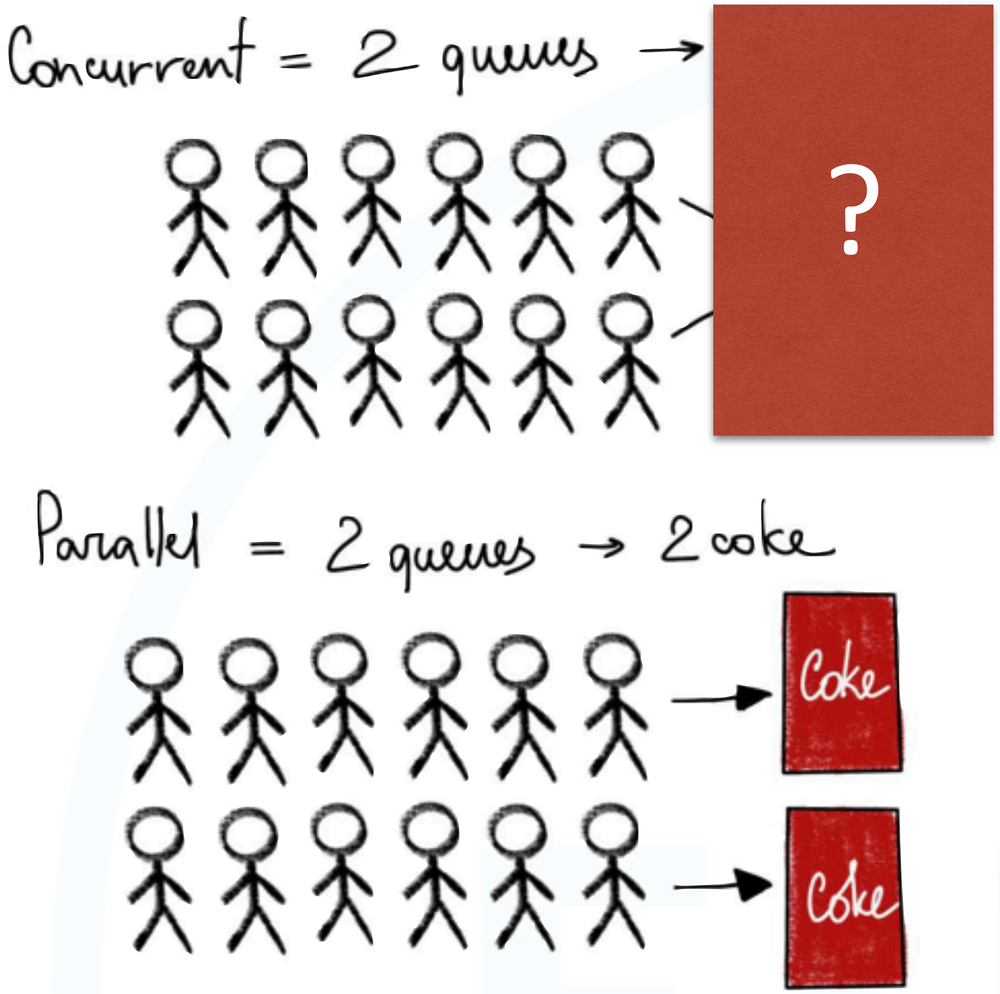
– **Coherent system** - users cannot perceive whether there are one or more elements underneath
  - Collaboration between the nodes necessary

– Note that nodes can be heterogeneous, and there are no assumptions on performance or size (IoT, HPC, ...)

Why do we need it?

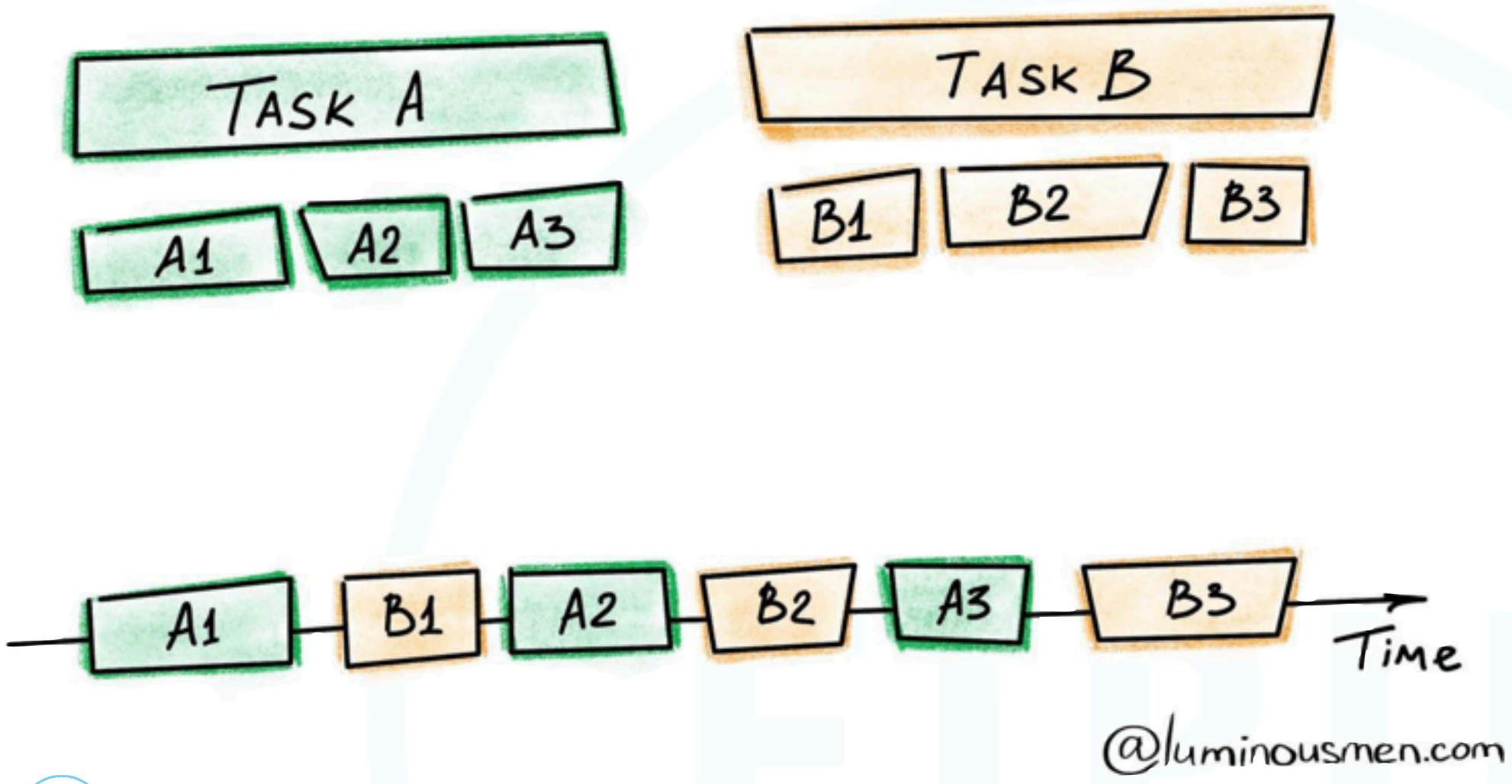
- Scaling up Facebook
  - 2004: Facebook started on a single server
    - Web server front end to assemble each user's page.
    - Database to store posts, friend lists, etc.
- 2008: 100M users
- 2010: 500M users
- 2012: 1B users
- 2019: 2.5B users

Concurrency vs Parallelism



© luminousmen.com

...


Task A
A1
A2
A3
Task B
B1
B2
B3
A1
B1
A2
B2
A3
B3
Time
@luminousmen.com

Concurrency

- The **ability** of different **parts or units of an algorithm** to be executed **out-of-order** or in **partial order without affecting** the **final outcome**
- Concurrency refers to the decomposability of a program, algorithm, or problem into order-independent or partially-ordered components or units of computation

DEF

Concurrency

- **Concurrency** is the execution of **more than one** task in overlapping periods
- An important detail is that tasks are not necessarily performed simultaneously (but it's possible)
- That is based on the notion of **Interruptability** — tasks can be divided into smaller and alternating subtasks

Parallelism

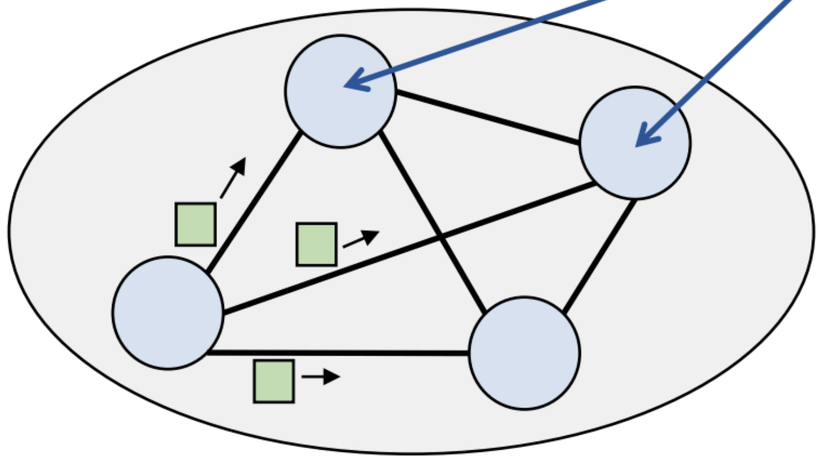
- **Parallelism** is the **simultaneous execution** of tasks
- Parallelism is a **subclass** of **concurrency**
  - There is concurrency without parallelism, but not the other way around
  - **Concurrency** allows for parallel execution of the concurrent units, which can significantly improve the overall speed of the execution in multi-processor and multi-core systems

In a nutshell

- **Concurrency** is about **dealing** with lots of things at once but **parallelism** is about **doing** lots of things at once



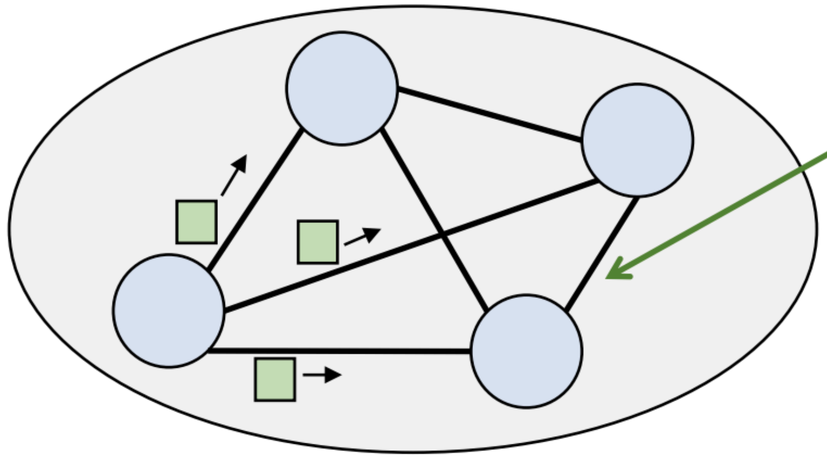
Challenges of DistSys



Multiple computers

- Concurrent execution.
- Independent failure.
- Autonomous administration.
- Heterogeneous.
- Large numbers.

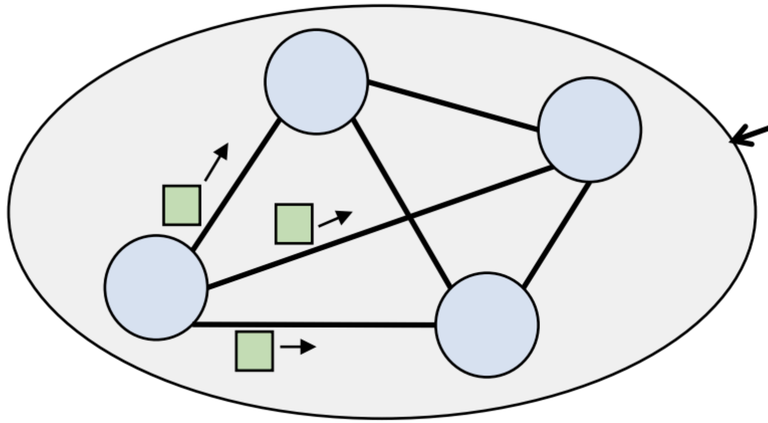
Challenges



Networked communication

- Asynchronous
- Unreliable
- Insecure

Challenges



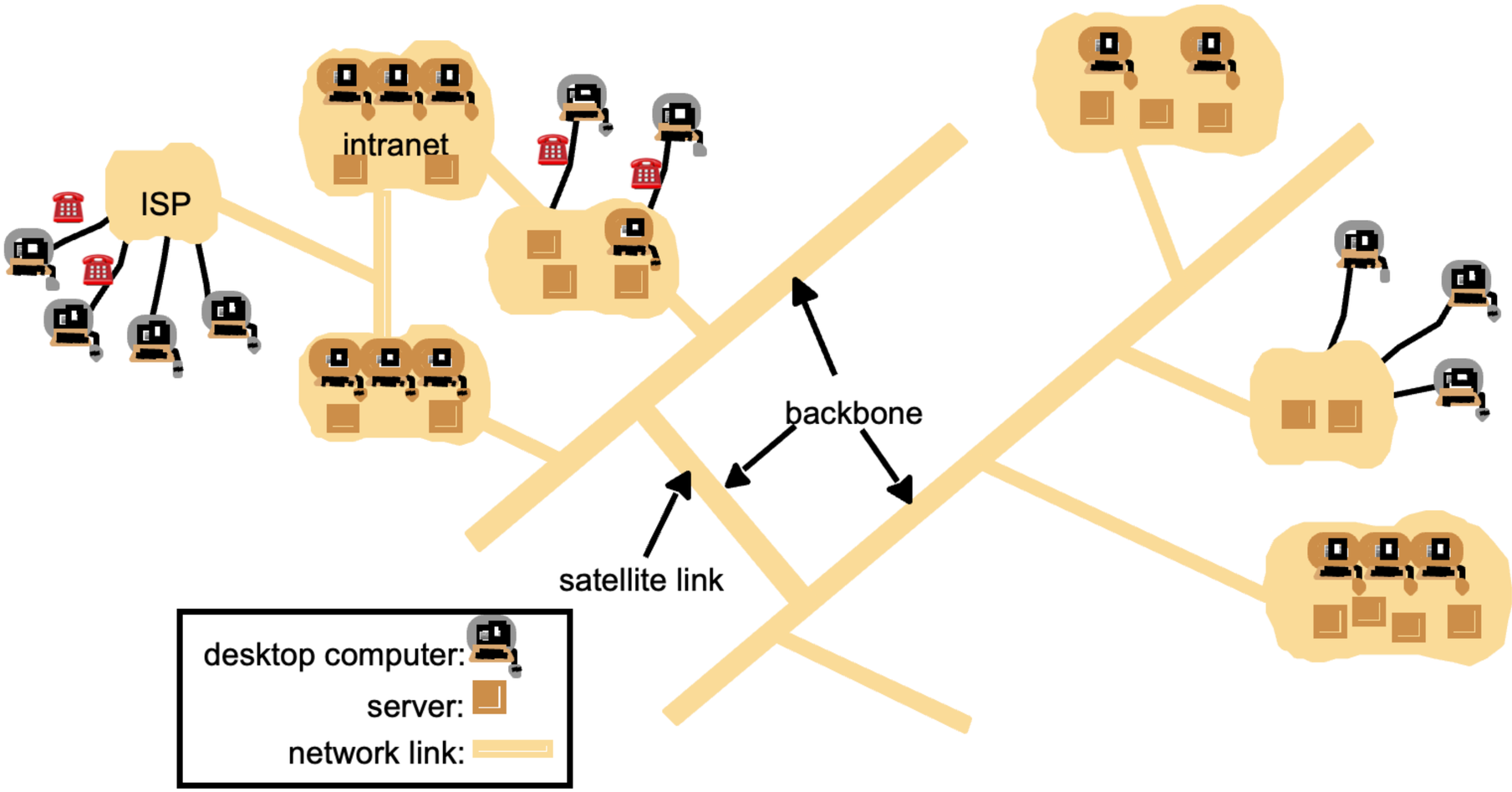
Common goal

- Consistency
- Transparency

Examples

Finance and commerce	eCommerce e.g. Amazon and eBay, PayPal, online banking and trading
The information society	Web information and search engines, Wikipedia; social networking: Facebook and MySpace.
Creative industries and entertainment	online gaming, music and film in the home, user-generated content, e.g. YouTube, Netflix
Healthcare	health informatics, on online patient records, monitoring patients
Education	e-learning, virtual learning environments; distance learning
Transport and logistics	GPS in route finding systems, map services: Google Maps, Google Earth
Science	The Grid as an enabling technology for collaboration between scientists
Environmental management	sensor technology to monitor earthquakes, floods or tsunamis

… even the Internet itself



DEF

Back to definition

A distributed system is a **collection** of autonomous computing elements that appears to its users as a single **coherent** system

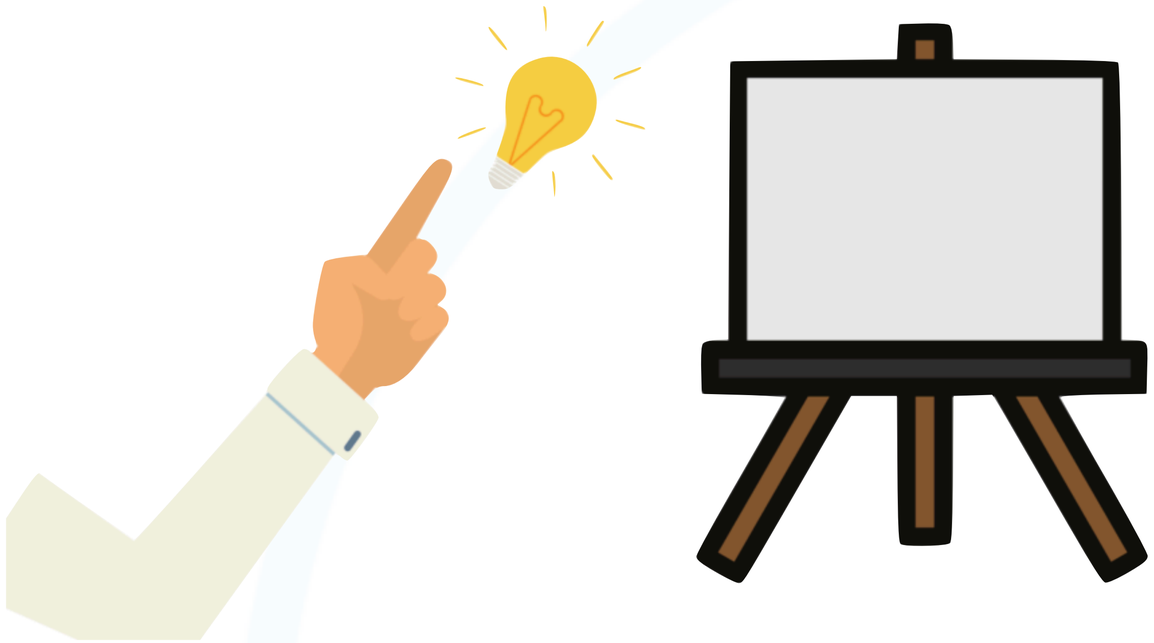
Characteristic 1: Collection …

- … of autonomous computing elements
- **Principle**: nodes can act independently from each other
- But in practice, nodes are programmed to achieve a common goal
- **Independence**: each node has its notion of **time**; we cannot assume a **global clock**
- We may also need to deal with **membership** → who can be a part of the system?
- How to organize such a collection? → very often as an **overlay network** (through TCP/IP or UDP channels)
  - Structured (tree/ring - well defined neighbors)
  - Unstructured (random neighbors)

Characteristic 2: Single coherent system

- Since that is hard, at least appear as **coherent**: behaving according to expectance of the users
- The end-user should **not be able to**:
  - Differentiate where the data is located
  - Differentiate on which process the execution takes place
  - **Distribution transparency**
- There are **trade-offs:**
  - **Sometimes the system will inevitably fail**

Design goals



Important design goals

- We don't need a distributed system for everything, but when we build one, we should **strive for**:

G.1 Support for **resource sharing**

G.2 Distribution **transparency**

G.3 Being open - **openness**

G.4 **Scalability**

G.1 Resource sharing

- What is a resource?
  - Peripherals, storage, data, files, services, infrastructure ...
- Why share?
  - Costs: high-end reliable resources (reuse, effectiveness)
- Common goal
  - P2P, BitTorrent, Blockchain networks, ...
- Good examples
  - Cloud resources (AWS S3, EC2, ...) - **IaaS**, **PaaS**, **Saas** **(infrastructure, platform, software - as a service)**

G.2 Distribution transparency

- Hide the distribution of resources from the users
- What is the optimal degree of transparency?
  - Full transparency comes at a great cost (hot replicas, ...)
  - Physical constraints (for start!)
    - e.g. **latency&throughput**
  - Sometimes the user does not want transparency!
    - Going offline on YouTube, DropBox, Google Drive...

Important transparencies

Access transparency hides the differences in data representation and how an object is accessed.

Location transparency enables resources to be accessed without knowledge of their physical or network location (URLs, IPFS).

Concurrency transparency enables several processes to operate concurrently using shared resources without interference. Leaving resources in consistent states (e.g. transactions).

Replication transparency enables multiple instances of resources to be used to increase reliability and performance without knowledge of the replicas by users or application programmers.

https://www.cl.cam.ac.uk/~jac22/books/ods/ods/node18.html

...continued

Failure transparency enables the concealment of faults, allowing users and application programs to complete their tasks despite the failure of hardware or software components.

Mobility/migration transparency allows the movement of resources and clients within a system without affecting the operation of users or programs (e.g. cloud computing, mobile networks).

Performance transparency allows the system to be reconfigured to improve performance as loads vary.

Scaling transparency allows the system and applications to expand in scale without change to the system structure or the application algorithms.

G.3 Openness

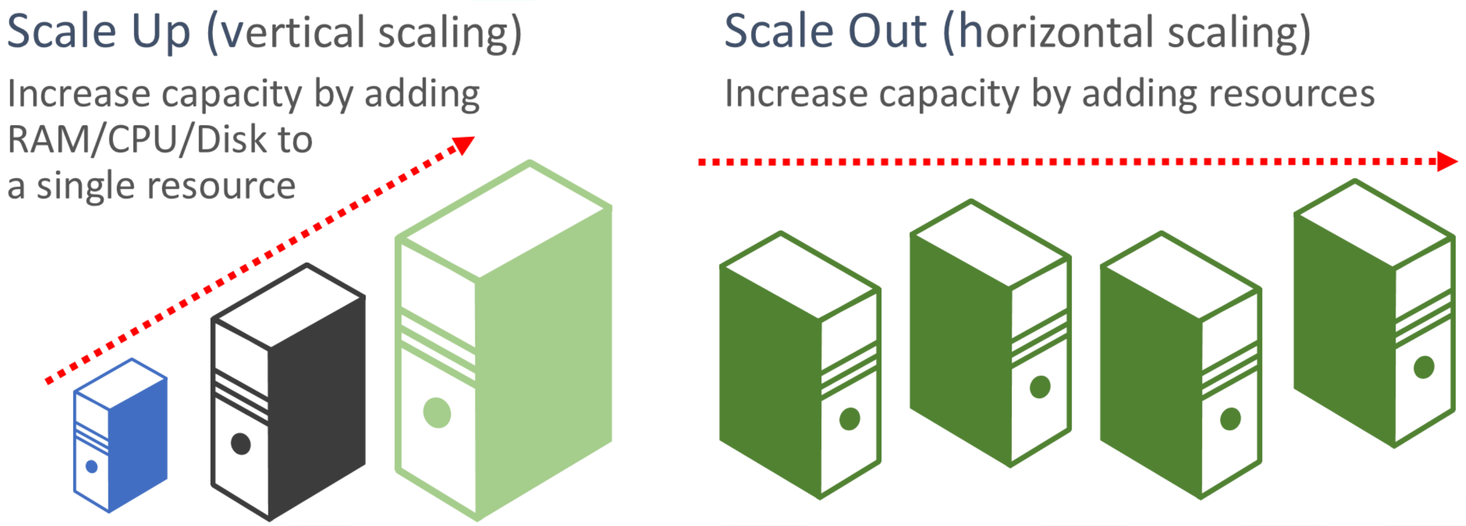
- Adhering to the standard rules that describe the syntax and semantics of individual components
- Conforming to well-defined **interfaces**
- Example: IDL - interface description language, REST services, ...
- **Policy** vs **mechanism**
  - Example: **Caching** is a mechanism
  - How to cache is a **policy**
    - Where, How much, Shared or private, Refreshing?

G.4 Scalability

- Dimensions of scalability
  - **Size** - easily adding more users or resources without noticeable loss of performance
    - Computational, storage, and networking limits
  - **Geographical** - the system can be spread geographically with no apparent latency issues
    - Sync vs. Async communication, reliability
  - **Administrative** - degree of how easily the system can be managed even if it spans many independent organizations
    - e.g. how to share a large radio telescope as a large-scale shared sensor network, security, etc.,
    - P2P as a good example

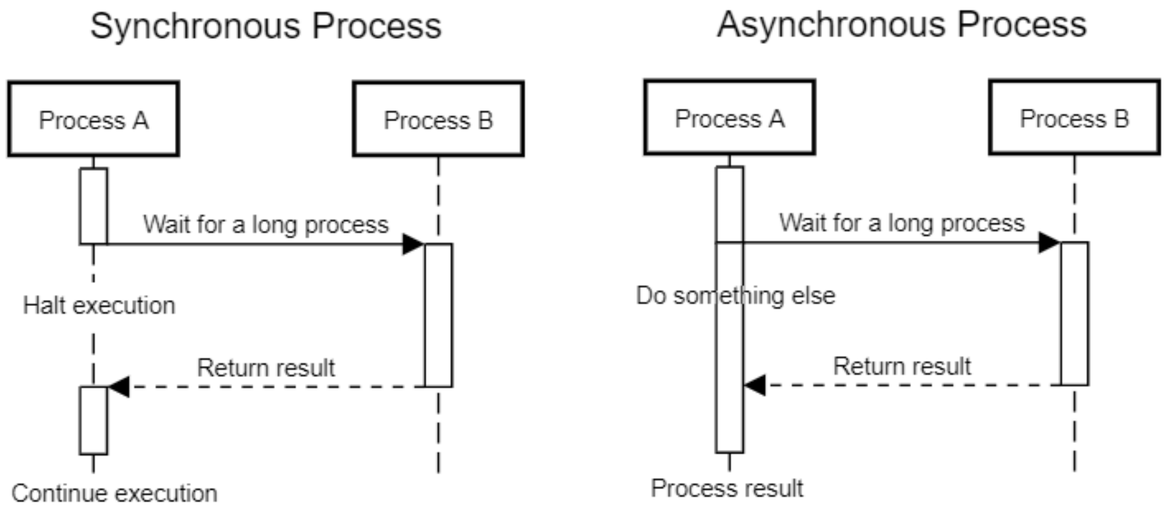
How to scale?

- **Scaling up** - increasing the capacity of existing components
- **Scaling out** - deploying more components



How to scale?

- Hide communication latencies
  - Try to avoid waiting for remote responses as much as possible
  - Switch **synchronous** communication with **asynchronous**



Geo-scalability problems

- Hard to switch from LAN to WAN
  - hard to keep synchronous interactions → latency increase
  - WAN links are much more unreliable
  - No **multipoint communication** (broadcasting)
    - Developing **naming** and **directory** services (with scaling problems of their own)

🔥 How to develop a **bad** system

- False (and often hidden) assumptions:
  - Network is reliable
  - Network is secure
  - Network is homogeneous
  - Topology does not change
  - Latency is zero
  - Bandwidth is infinite
  - Transport cost is zero
  - There is one (always available) administrator





Distributed systems

Types of distributed systems



Distributed systems types

T1 **Distributed computing system** - high-performance computing (HPC)

– Dedicated expensive hardware in solving difficult (often scientific) problems

T2 **Distributed information systems**

– Your everyday centralized, but still distributed system

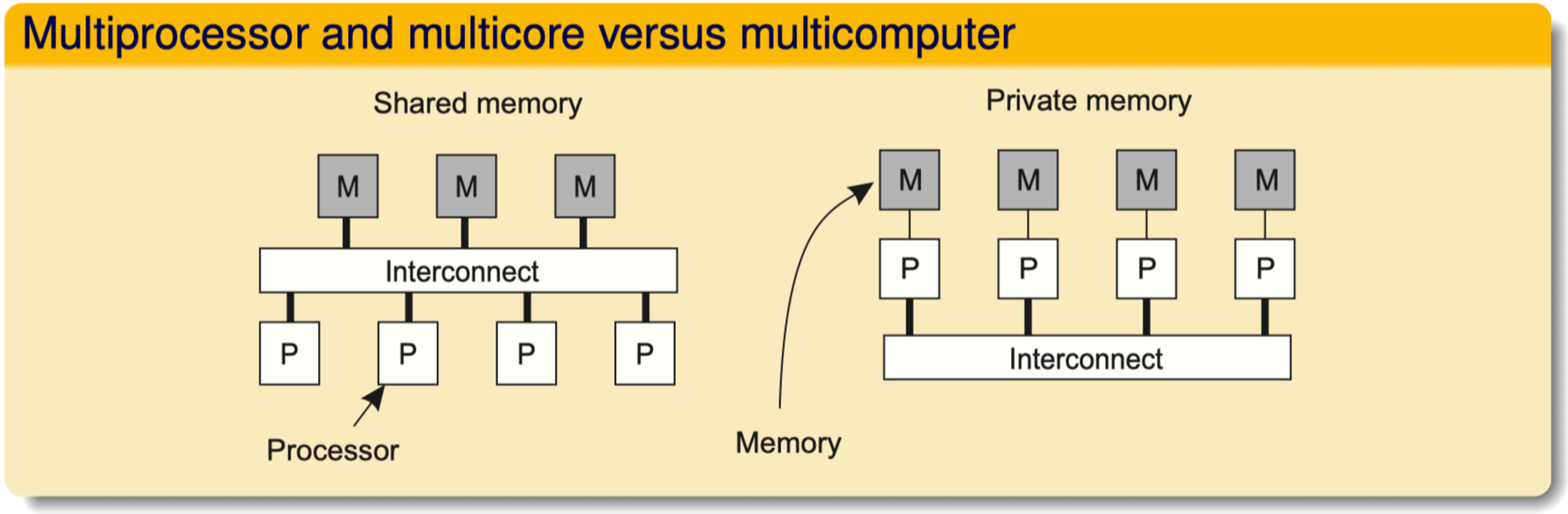
T3 **Distributed pervasive computing**

– Often decentralized and distributed

T1

HPC

- High-performance computing
  - started with parallel computing



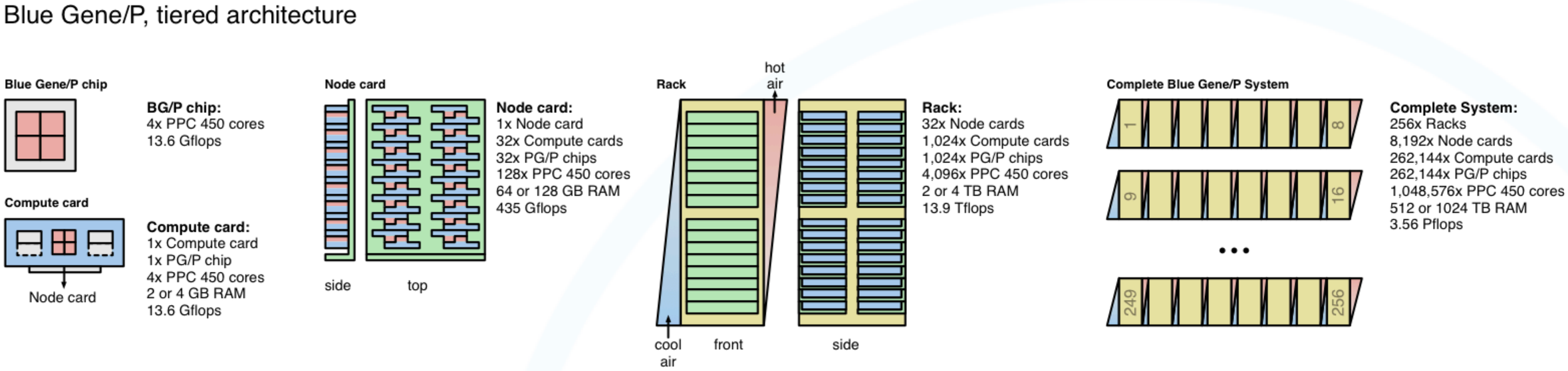
Shared vs. Private memory

- Multiprocessors (w. shared-memory) are relatively **easy to program** in comparison to multi-computer approaches
- Problems arise with more cores
- There are "shared-memory" models atop multi-computer approaches
  - Example: Map all main-memory pages (from different processors) into one **single virtual address space.** If the process at processor A addresses a page P located at processor B, the OS at A traps and fetches P from B, just as it would if P had been located on a local disk.
  - Failed to meet the expectations → **abbandoned**

Example: IBM Blue Gene



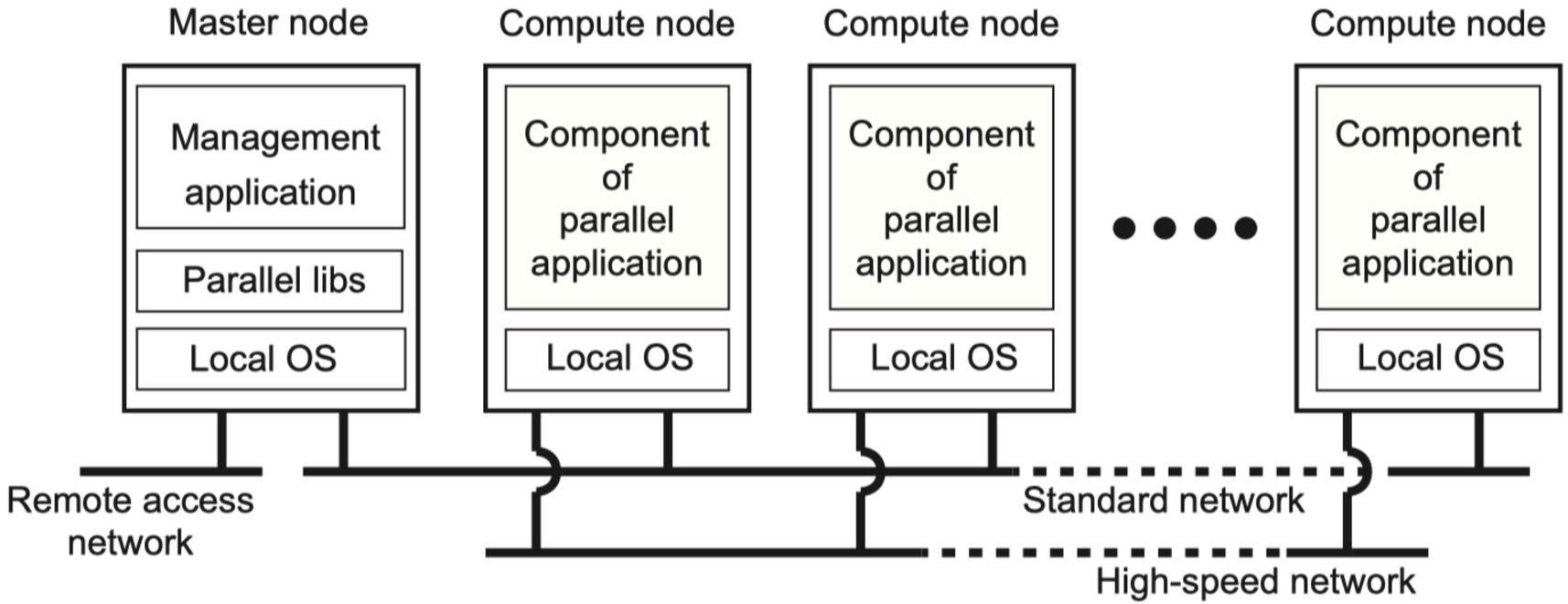
IBM Blue Gene P



- Cosmology, fluid-dynamics, ...

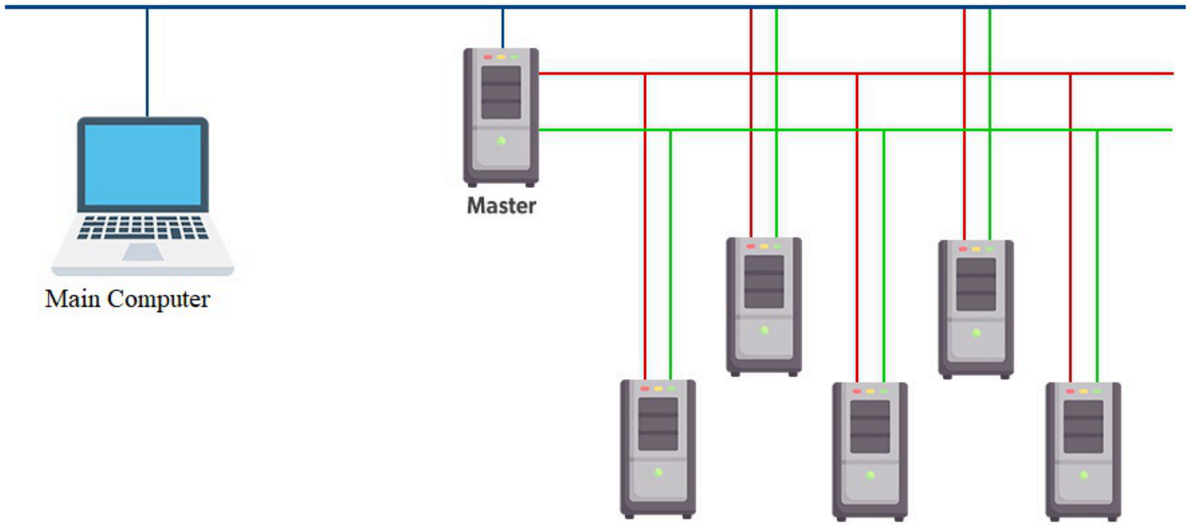
Also HPC: Cluster computing

- A group of high-end systems connected via LAN
  - Homogeneous group (easy to plan/execute programs)
  - Single managing node (centralised) - e.g. **Apache Spark**



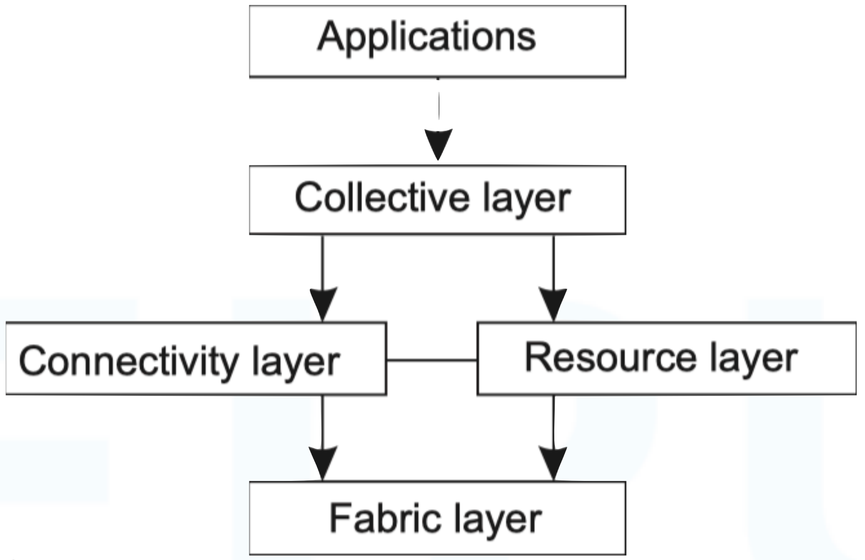
Key characteristic

- Low latency, high bandwidth interconnection
- Usually master-slave (controller-responder)



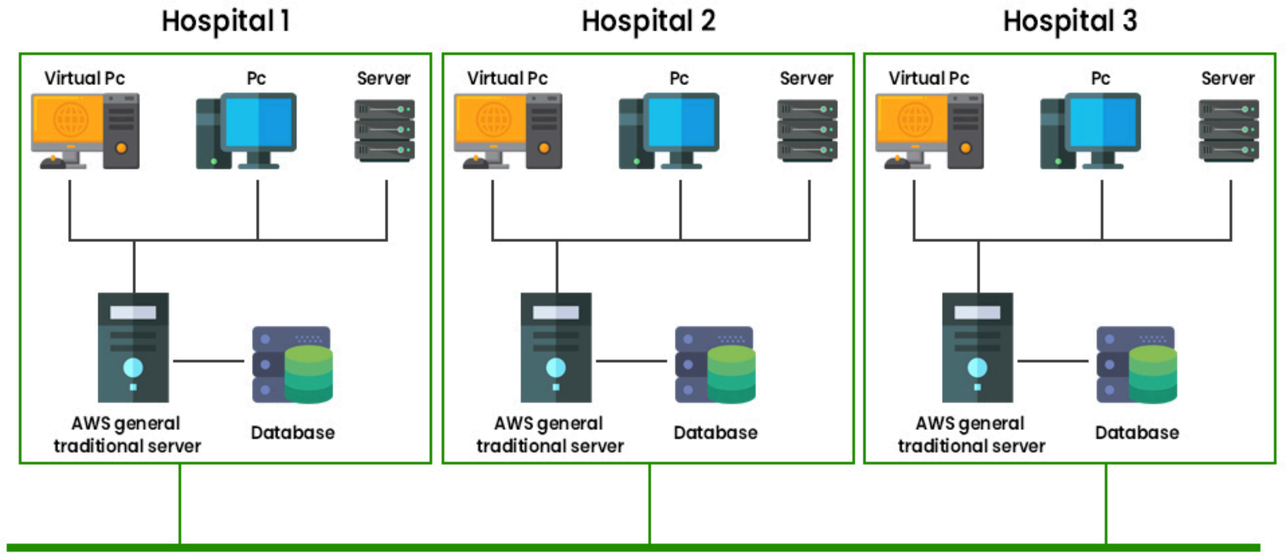
Grid computing

- Homogeneous > **heterogeneus**
- **Several organizations** often on **WAN** (wide-area network)
- Again often used in scientific purposes
  - Particle physicists
  - Astronomy
  - Medicine

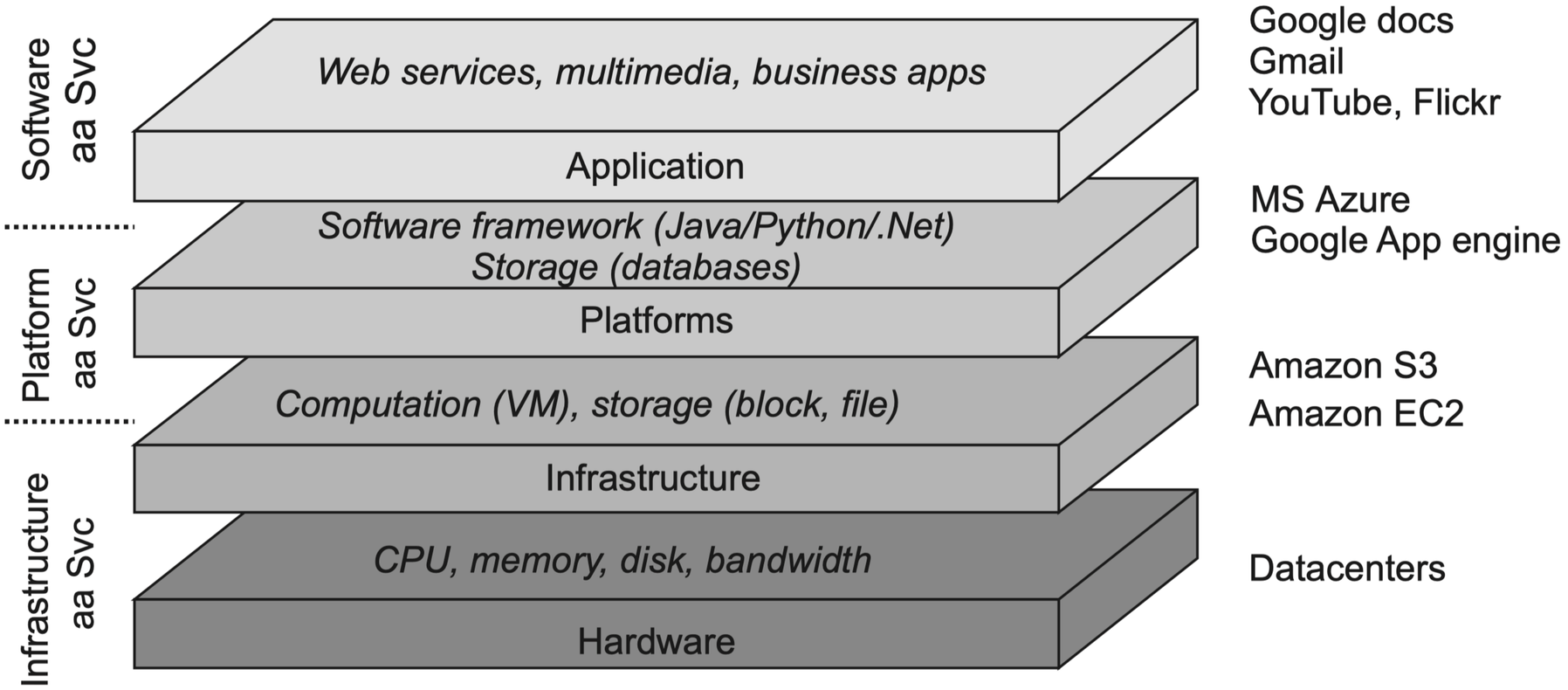


Key characteristic

- WAN, multiple organizations



Cloud computing



Cloud computing

- **Hardware**: processors, routers, power management, cooling, security systems - customers don't get to see these
- **Infrastructure:** virtualization - allocating virtual servers, networks and storage devices
- **Platform:** even higher-level abstractions, e.g. for storage - example: **Amazon S3, RDS, Dynamo...** or computing: **Amazon Lambda,** or both: **Google Firebase**
- **Application:** actual applications - GoogleMail/Docs, Dropbox, ...

Is cloud computing cost-effective?

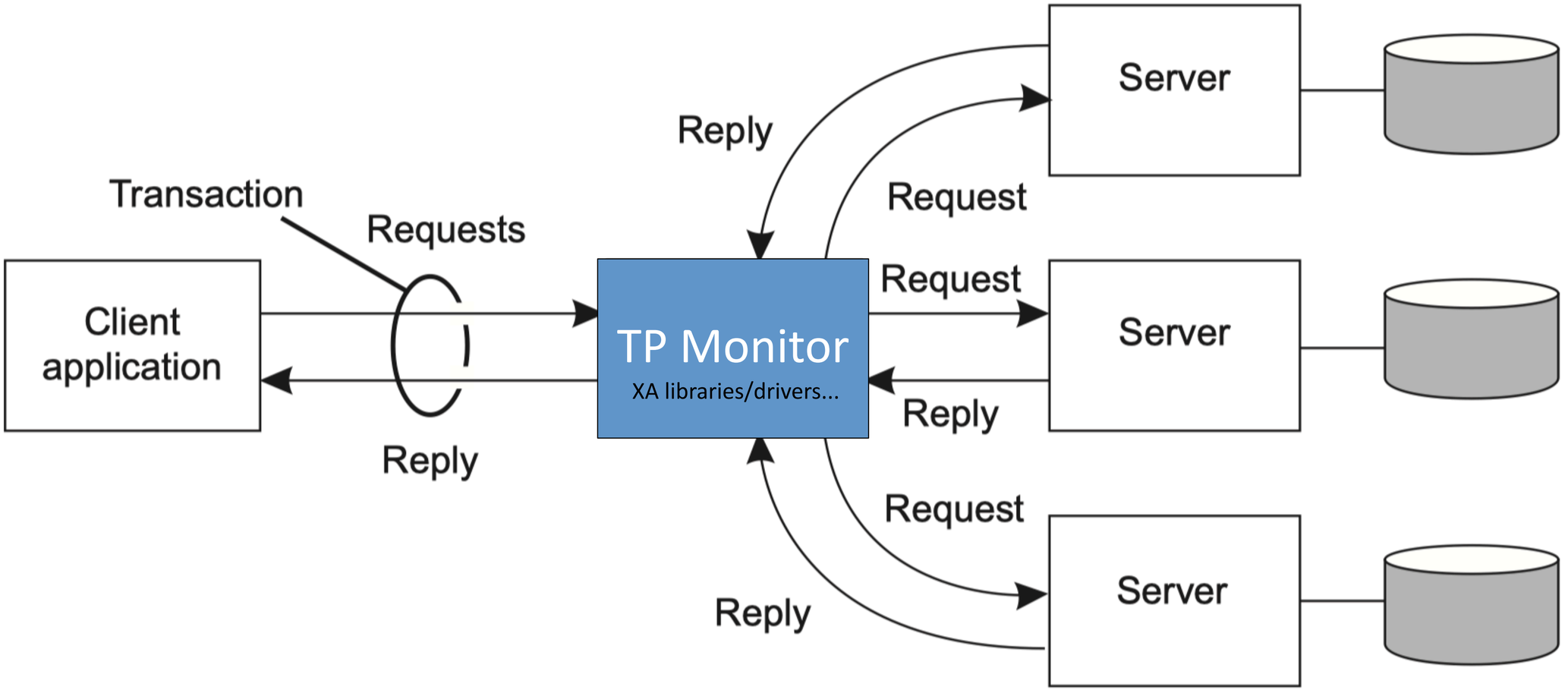
- Is outsourcing cheaper for organizations?
- Costs:
  - Storage
  - Computing
  - Platforms
  - Bandwidth
  - SLA's...
- OPEX vs CAPEX
- Scaling considerations

T2

Distributed information systems

- Organizations had many networked applications, but the challenge was in interoperability
  - Networked application example: client-server architecture
- How to communicate application-to-application?
  - **Enterprise Application Integration**
    - one possible solution
    - **Distributed transactions** are challenging

TPM: transaction processing monitor

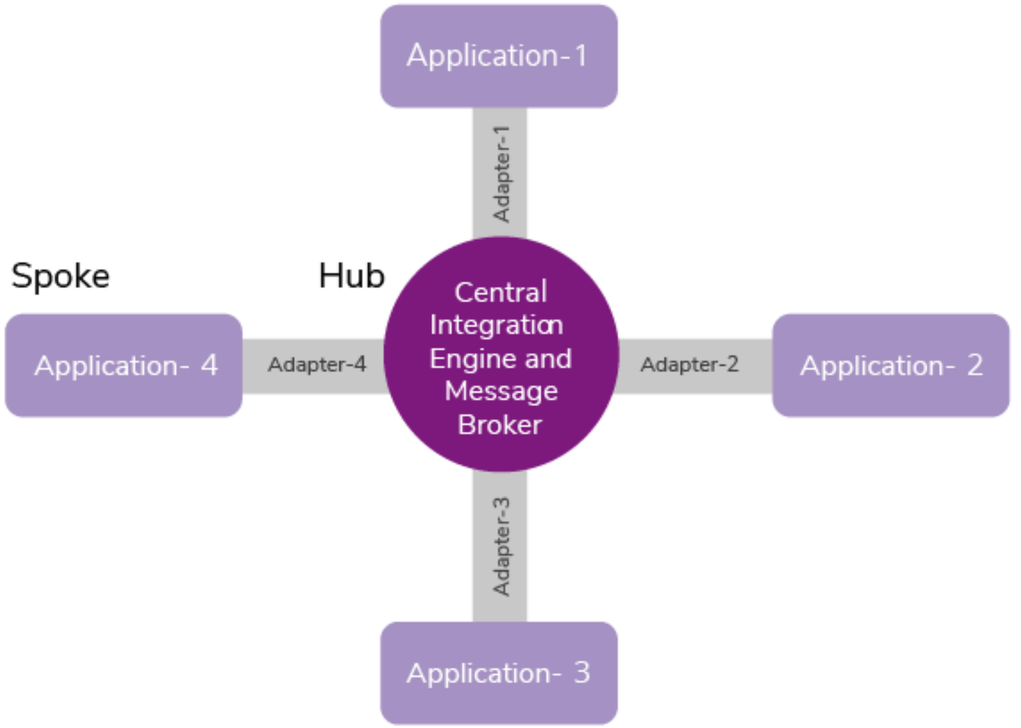


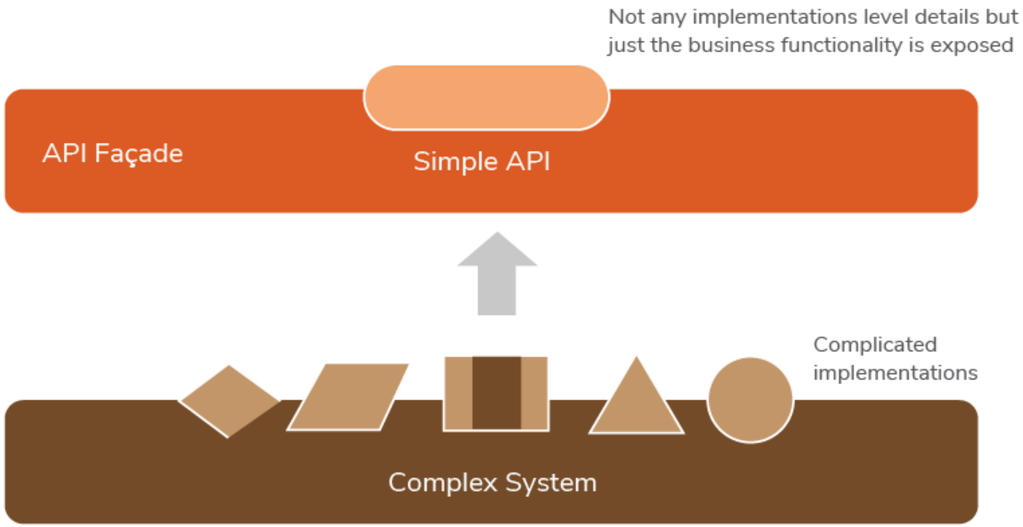
Enterprise Application Integration

- **Purpose**
  - **Data integration**: Ensures that information in multiple systems is kept consistent
  - **Vendor independence**: Extracts business policies or rules from applications and implements them in the EAI system so that even if one of the business applications is replaced with a different vendor's application, the business rules do not have to be reimplemented
  - **Common facade**: An EAI system can front-end a cluster of applications, providing a single consistent access interface to these applications and shielding users from having to learn to use different software packages

Enterprise Application Integration

- **Patterns**
  - **Mediation (intra-communication): broker** between multiple applications. The broker propagates the changes within relevant applications
  - **Federation (inter-communication):** In this case, the EAI system acts as the overarching **facade** across multiple applications. All event calls from the 'outside world' to any of the applications are front-ended by the EAI system


Application-1
Adapter-1
Spoke
Hub
Central Integration Engine and Message Broker
Application- 4
Adapter-4
Adapter-2
Application- 2
Adapter-3
Application- 3


Not any implementations level details but just the business functionality is exposed
API Façade
Simple API
Complicated implementations
Complex System

Other ways to integrate

- File transfer 
  - Not very flexible
- Shared databases 
  - Risk of bottleneck
- RPC
  - Effective, could be hard to maintain
  - The caller and Receiver should be up&running
- Messaging
  - Enabling async communication 
  - **Decoupling in time and space**

T3

Distributed pervasive systems

- Next generation systems in which nodes are **small**, **mobile** in (often embedded) systems that **naturally blend** in the user's environment (IoT)
- Sub-types (overlapping):
  - **Ubiquitous computing systems** - pervasive and continuously present (in interaction with users)
  - **Mobile computing systems** - inherently mobile
  - **Sensor (and actuator) networks** - sensing and actuation in the environment

Ubiquitous systems

- Core elements:
  - **Distribution:** Devices are networked, distributed, and accessible in a transparent manner
  - **Interaction:** Interaction between users and devices is highly unobtrusive
  - **Context awareness:** The system is aware of a user's context to optimize interaction
  - **Autonomy:** Devices operate autonomously without human intervention and are thus highly self-managed
  - **Intelligence:** The system as a whole can handle a wide range of dynamic actions and interactions

Mobile computing

- Key challenges:
  - **Discovery:** since the location changes over time, how can we establish a network? Who are my peers?
  - **Connectivity:** no stable routes, but also no guaranteed connectivity → **disruption-tolerant** networking
- Example strategy:
  - **pocket-switched network**
    - Alice maintains a list of friend devices
    - Alice sends the message to Bob via friends only

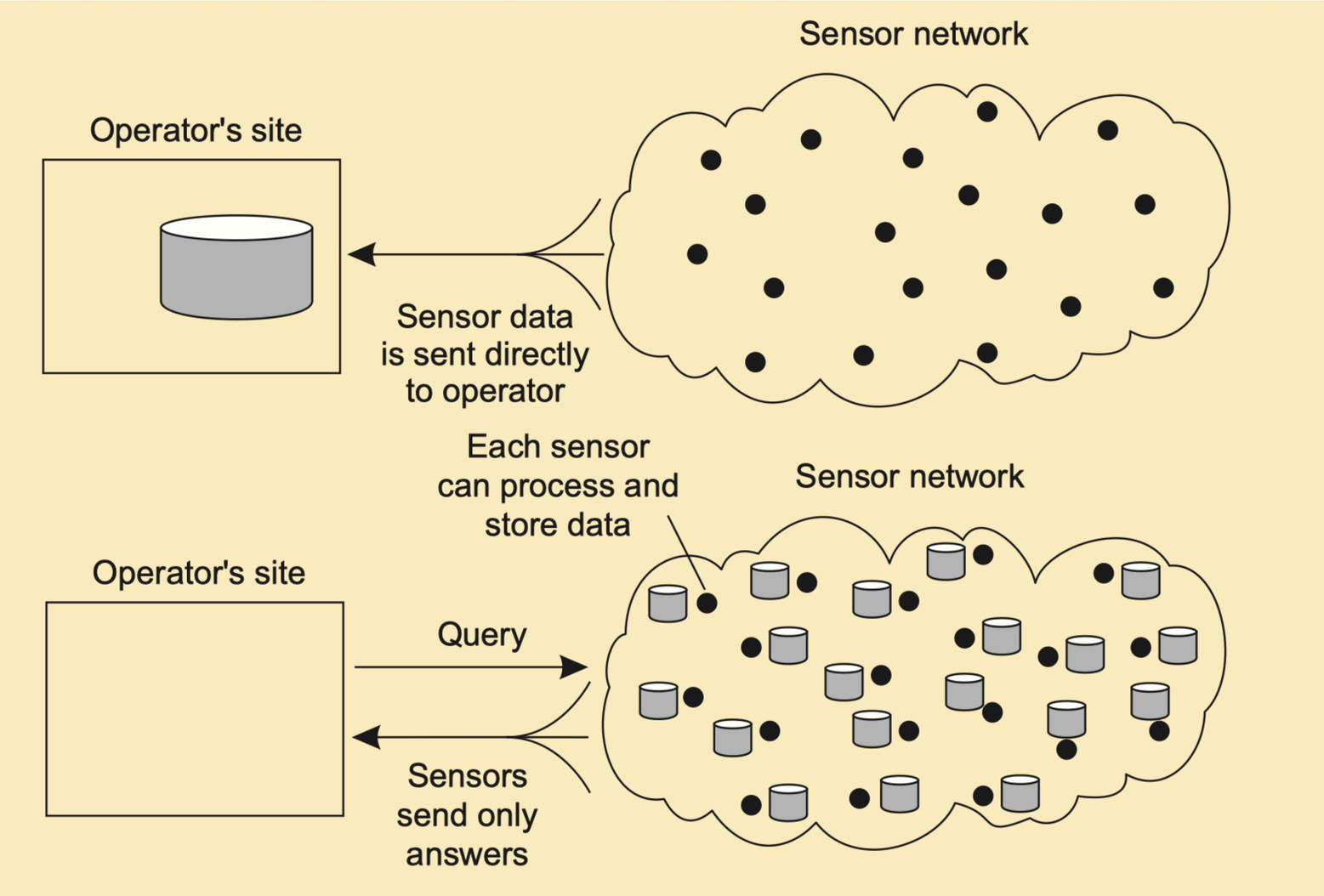
Sensor networks



Sensor networks

- Typically many nodes: 10s - 1000s
- Simple hardware (small memory/compute/ communication capacity - saving power)
- Battery-powered (or even battery-less)

Two extremes



Energy conservation

- Using **duty cycles**
  - A node is active during T_{active} active time units, and then suspended for $T_{suspended}$ units, to become active again. Duty cycle τ is expressed as:

$$\tau = \frac{T_{\text{active}}}{T_{\text{active}} + T_{\text{suspended}}}$$

  - Typically 10-30%, but as low as 1%
  - Challenge: peer discovery and **synchronization**

Fireflies



Why does IoT keep failing?

- Security is hard to do right
  - Huge attack surface
- Embedded devices hard to program
  - C still dominant
  - Everything is distributed
- Outdated software
  - Hard to do upgrades
- Lack of standards
  - So many vendors

Self-assesment

How do I define a distributed system?

What are the main challenges in designing one?

Which types of DS can I differentiate?

What are the important design goals?

What types of cloud computing are there?